CSE 554: Assignment 3

Jack Lowry, Nam Pho, Brett Saiki

March 2, 2026

1 General Matrix Multiplication

Q1. (a) What is the operational intensity of a GEMM in terms of M, N, K dimensions?

Consider the GEMM operation $AB = C$ for matrices A, B, and C. The matrices are of sizes $(M \times K)$, $(K \times N)$, and $(M \times N)$. Each element c_{ij} of matrix C is a dot product requiring K multiplications and $K - 1$ additions. This is replicated for all MN elements.

$$W_{FLOPs} = MN\,(2K - 1)$$

For the same GEMM set up to read A and B then write C from memory requires moving the number of elements in each as the product of their dimensions. This value needs to be scaled by the byte multiple of the data type size (S). For example, $S = 2$ for FP16 representation of all elements and assuming all elements are of the same data type.

$$Q_{Bytes} = S\,(MK + KN + MN)$$

The operational intensity (I) is defined as the ratio of work (W) in FLOPs to the memory traffic (Q) as measured in Bytes.

$$I = \frac{W}{Q} = \boxed{\frac{MN\,(2K-1)}{S\,(MK + KN + MN)}} \text{ FLOPs per Byte}$$

(b) Calculate the compute intensity for M in 128 to 2048 with step 128 and $(N, K) = (512, 512), (4096, 4096), (14336, 4096), (4096, 1024), (1024, 4096)$ and show the result in a table.

See Table 1 below.

Table 1: Operational Intensity (I) for various GEMM dimensions.

	(N, K) Pairs				
M	(512, 512)	(4096, 4096)	(14336, 4096)	(4096, 1024)	(1024, 4096)
128	170.50	240.91	246.08	221.30	221.38
256	255.75	455.06	473.86	389.90	390.05
384	306.90	646.66	685.30	522.64	522.83
512	341.00	819.10	882.11	629.85	630.08
640	365.36	975.12	1065.74	718.25	718.51
768	383.62	1116.95	1237.49	792.39	792.68
896	397.83	1246.46	1398.46	855.46	855.78
1024	409.20	1365.17	1549.65	909.78	910.11
1152	418.50	1474.38	1691.91	957.04	957.39
1280	426.25	1575.19	1826.02	998.54	998.90
1408	432.81	1668.54	1952.65	1035.26	1035.64
1536	438.43	1755.21	2072.42	1068.00	1068.39
1664	443.30	1835.91	2185.87	1097.36	1097.76
1792	447.56	1911.23	2293.48	1123.84	1124.25
1920	451.32	1981.69	2395.70	1147.85	1148.27
2048	454.67	2047.75	2492.91	1169.71	1170.14

(c) What is CUTLASS? What is cuBLAS? What is the key difference between them?

CUTLASS and cuBLAS are both libraries for high-performance GEMM and deep learning operations. CUTLASS is an open-source C++ template library built on top of CUDA emphasizing customization. It provides specialized building blocks to can be assembled together into efficient kernels with expert knowledge. On the other hand, cuBLAS is a proprietary, pre-compiled library that is intended for off-the-shelf use.

Q2. (a) Profile the cuBLAS `cublasGEMMEx` performance for the above shapes and plot the performance. Plot one figure for one shape with the M dimension on the x-axis and TFLOPs (total compute/time) on the y-axis. Please use the profiling methods in `Section1/prof_example.cu`. We do warmups and L2 cache flush to increase the profiling accuracy. The example plotting script is given at `Section1/plot_gemm.py`.

See next section below for all five combined plots.

(b) Profile the CUTLASS GEMM performance using the CUTLASS profiler for the above shapes. The CUTLASS profiler will iterate through all kernel implementations with different tile sizes and configurations. Use `--profiling-iterations` of 100. Profile different `--split_k_slices=1,2,4,8` with `--split_k_mode=serial`. Split K will control the number of splits in K dimension. For example, for GEMM shape $(128, 1024, 4096)$, when split K is 4, the kernel will first treat it as 4 GEMM

problems of size $(128, 1024, 1024)$ and perform a reduction to add the partial results. Take the best performance across all implementations for each GEMM shape. Plot the CUTLASS performance in the same figure as cuBLAS.

Below are five plots for cuBLAS vs CUTLASS GEMM tests for M and K combinations $(N, K) = (512, 512), (4096, 4096), (14336, 4096), (4096, 1024), (1024, 4096)$ and varying M in 128 to 2048 with step 128. These are Figures 1, 2, 3, 4, and 5 below, respectively.

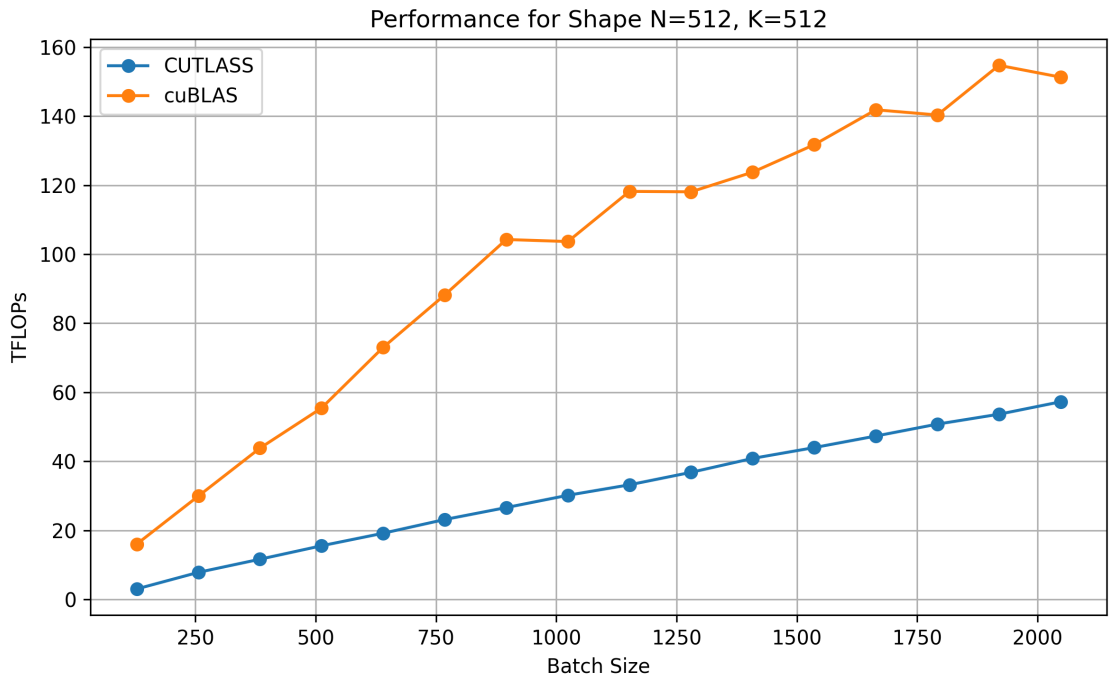

Figure 1: cuBLAS vs CUTLASS for a various batch size and $N = 512$ and $K = 512$.

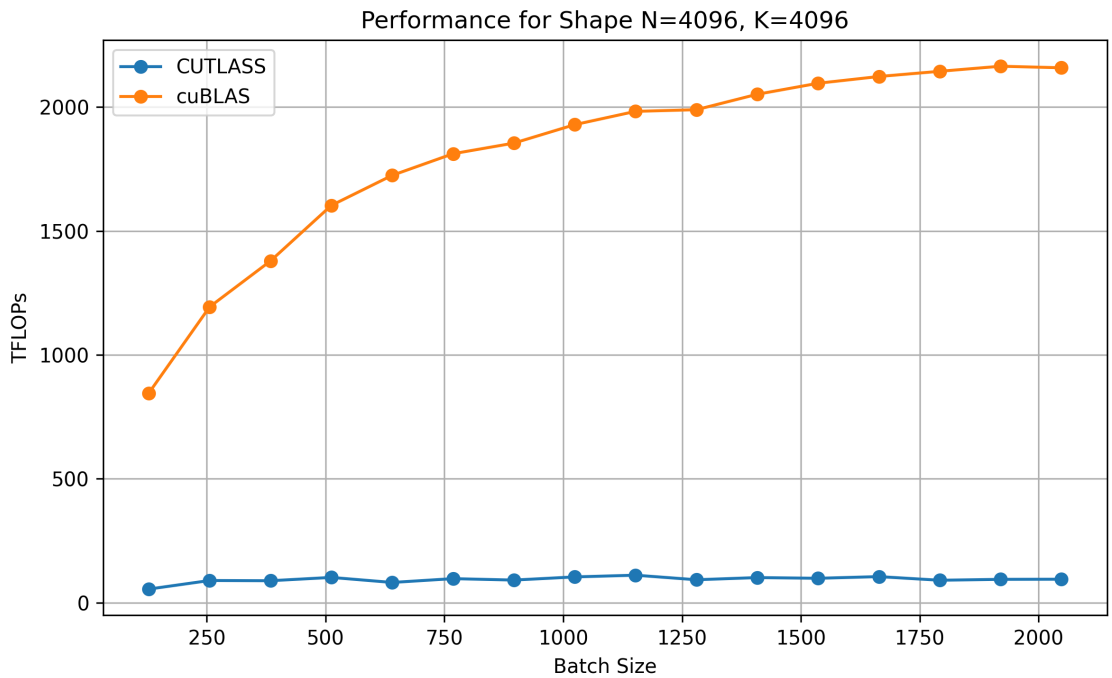

Figure 2: cuBLAS vs CUTLASS for a various batch size and $N = 4096$ and $K = 4096$.

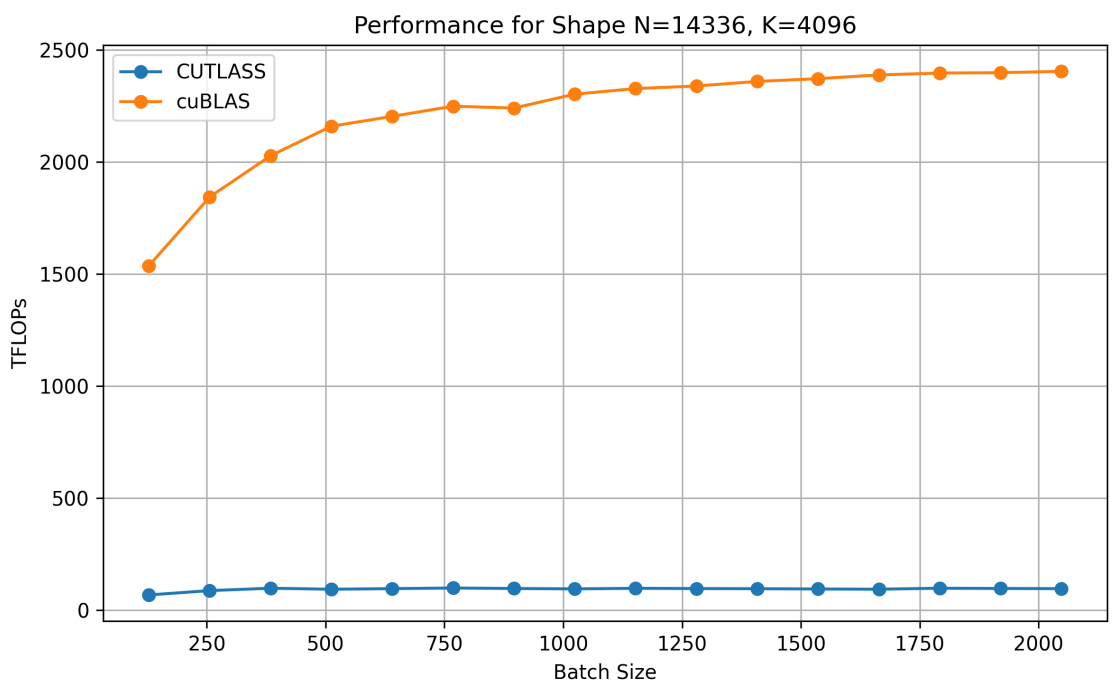

Figure 3: cuBLAS vs CUTLASS for a various batch size and $N = 14336$ and $K = 4096$.

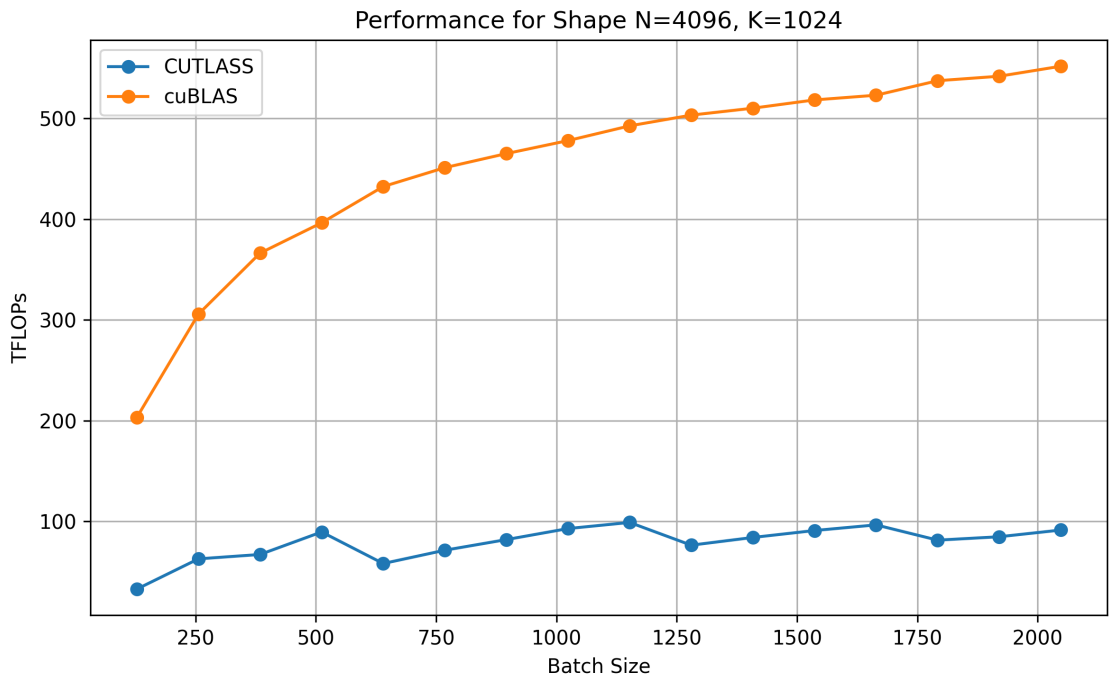

Figure 4: cuBLAS vs CUTLASS for a various batch size and $N = 4096$ and $K = 1024$.

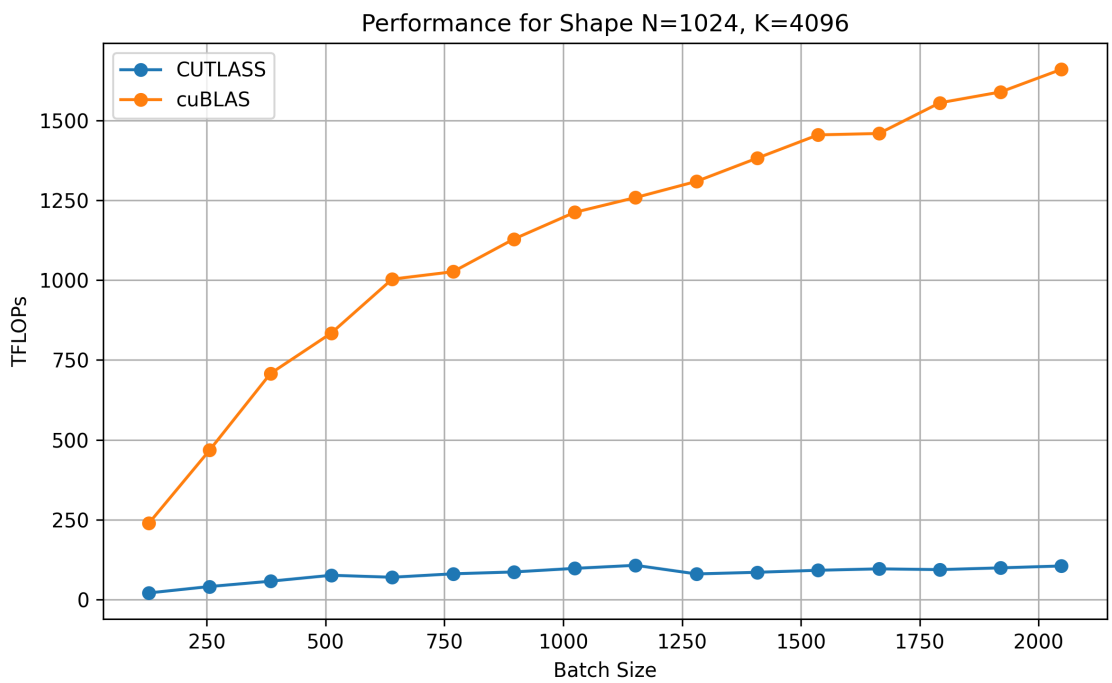

Figure 5: cuBLAS vs CUTLASS for a various batch size and $N = 1024$ and $K = 4096$.

(c) Investigate the kernel that achieves the best performance in CUTLASS. What is the relationship between problem size, tile size, and split K?

Maximum CUTLASS performance was achieved at 114.605 TFLOPs. The kernel was `cutlass_tensorop_h1688gemm_256x128_32x2_nn_align8`. The matrice dimensions were $M = 1152$, $N = 4096$, and $K = 4096$.

(d) What is the difference between the CUTLASS performance and the cuBLAS performance for different shapes? Describe your findings in a few sentences.

cuBLAS outperforms CUTLASS for GEMM operations. However, for smaller sized matrices the performance gap is relatively smaller than for bigger matrices. Put another way, CUTLASS and cuBLAS can perform similarly for small matrices but performance will begin to substantially diverge with cuBLAS outperforming CUTLASS more and more as the problem size increases.

2 Attention

Q1. (a) Given model `num_kv_heads`, `num_qo_heads`, `head_dim`, what is the operational intensity of: (1) prefill attention, when the prompt length is p and (2) decode attention, when the context length is c?

Let $H_{kv} =$ `num_kv_heads`, $H_{qo} =$ `num_qo_heads`, and $d =$ `head_dim`. We assume the weights are quantized into FP16 or BF16.

Encoding. For each Q head, we need to compute the attention which requires $2p^2d$ operations for $QK^\top$ and $2p^2d$ operations when computing the attention. Thus, we need $4H_{qo}p^2d$ operations for all heads. Finally, we need to compute the attention. To do so, we need to read in $H_{qo}pd$ entries for Q, and $H_{kv}pd$ entries each for K and V. Thus we need to read $2pd(H_{qo} + 2H_{kv})$ bytes in total, and the operational intensity of prefill attention is as follows.

$$I_{\text{prefill}} = \frac{4H_{qo}p^2d}{2pd(H_{qo}pd + 2H_{kv}pd)} = \frac{2H_{qo}p}{H_{qo} + 2H_{kv}}.$$

Decoding. For each Q head, computing the attention requires $2cd$ operations for $QK^\top$, and $2cd$ operations computing the attention. Thus, we need $4H_{qo}cd$ operations for all heads. To do so, we need to read in $H_{qo}d$ entries for Q and $H_{kv}cd$ entries each for K and V. Thus we need to read $2d(H_{qo} + 2H_{kv}c)$ bytes in total, and the operational intensity of decode attention is as follows.

$$I_{\text{decode}} = \frac{4H_{qo}cd}{2d(H_{qo} + 2H_{kv}c)} = \frac{2H_{qo}c}{H_{qo} + 2H_{kv}c}.$$

(b) Compute the operational intensity for llama3-1B, llama3-3B, llama3-8B for prefill attention, $p = 2^7 - 2^{15}$ and for decode attention, $c = 2^7 - 2^{15}$.

The results are contained in Table 2.

	LLaMA3-1B		LLaMA3-3B		LLaMA3-8B	
p / c	I_{prefill}	I_{decode}	I_{prefill}	I_{decode}	I_{prefill}	I_{decode}
128	170.67	3.94	153.60	2.97	170.67	3.94
256	341.33	3.97	307.20	2.98	341.33	3.97
512	682.67	3.98	614.40	2.99	682.67	3.98
1024	1365.33	3.99	1228.80	3.00	1365.33	3.99
2048	2730.67	4.00	2457.60	3.00	2730.67	4.00
4096	5461.33	4.00	4915.20	3.00	5461.33	4.00
8192	10922.67	4.00	9830.40	3.00	10922.67	4.00
16384	21845.33	4.00	19660.80	3.00	21845.33	4.00
32768	43690.67	4.00	39321.60	3.00	43690.67	4.00

Table 2: Operational intensity (FLOPs/byte) for prefill (I_{prefill}, varying prompt length p) and decode (I_{decode}, varying context length c) attention across LLaMA3 models. Decode intensity converges to H_{qo}/H_{kv} (4.0 for 1B/8B, 3.0 for 3B), confirming decode is always memory-bound. Prefill intensity grows linearly with p, becoming compute-bound at long sequences.

Q2. (a) Evaluate the prefill attention performance of one layer using `FlashInfer` and `torch.nn.functional.scaled_dot_product_attention` for batch sizes = 1 and $p = (2^7, 2^8, \ldots, 2^{15})$. Plot the compute utilization TFlops of each model. Use $\log_2(p)$ as the x-axis. Generate one subplot per model inside a shared figure. In each subplot, draw line plots for both torch SDPA and FlashInfer using different line styles or colors. Label axes clearly.

i. Batch size = 1, $p = 2^7 - 2^{15}$.

Our results are shown in Figure 6.

ii. Batch size = $2^0 - 2^6, p = 1024$ for all three models.

Our results are shown in Figure 7.

(b) Evaluate the decode attention performance of one layer using torch SDPA and FlashInfer. Plot memory bandwidth utilization (GB/s) similar to previous questions.

i. Batch size = 1, $c = 2^7 - 2^{15}$.

Our results are shown in Figure 8.

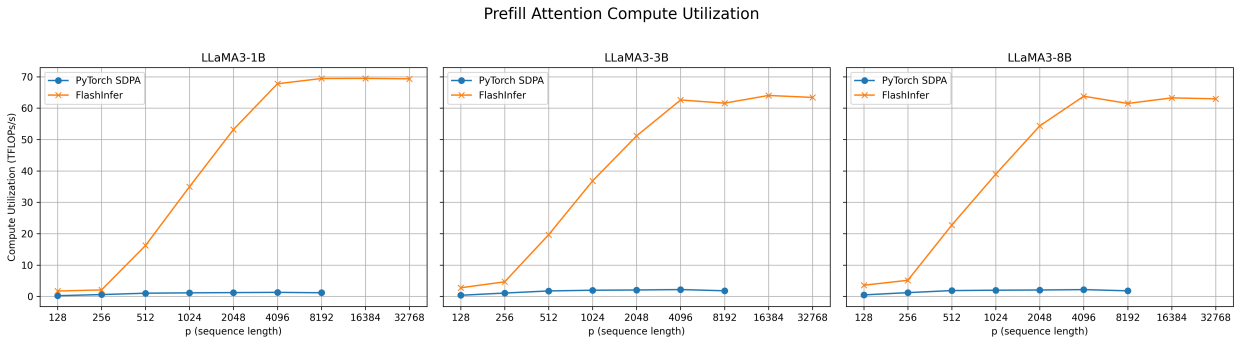

Figure 6: Compute utilization (TFLOP/s) for prefill attention for LLaMA3-1B, LLaMA3-3B, LLaMA3-8B for a batch size of 1 and sequence lengths between 2^7 to 2^{15}.

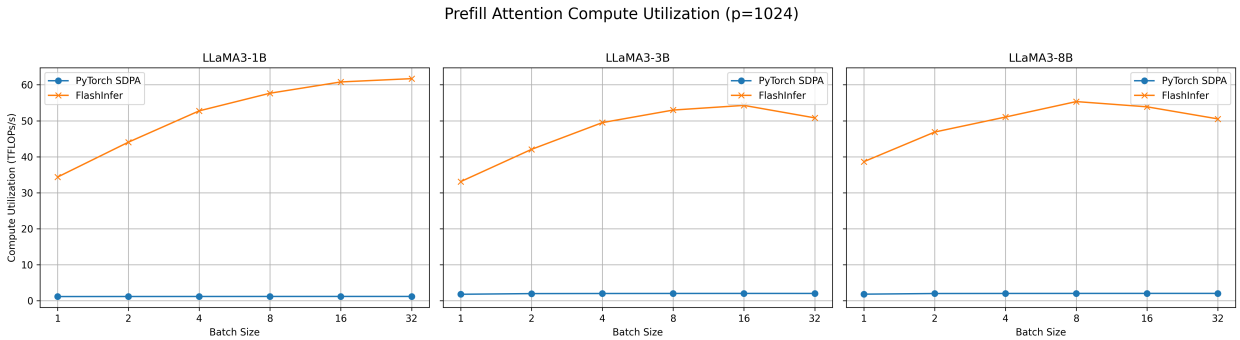

Figure 7: Compute utilization (TFLOP/s) for prefill attention for LLaMA3-1B, LLaMA3-3B, LLaMA3-8B for a batch sizes from 1 to 32 and a sequence length of 1024.

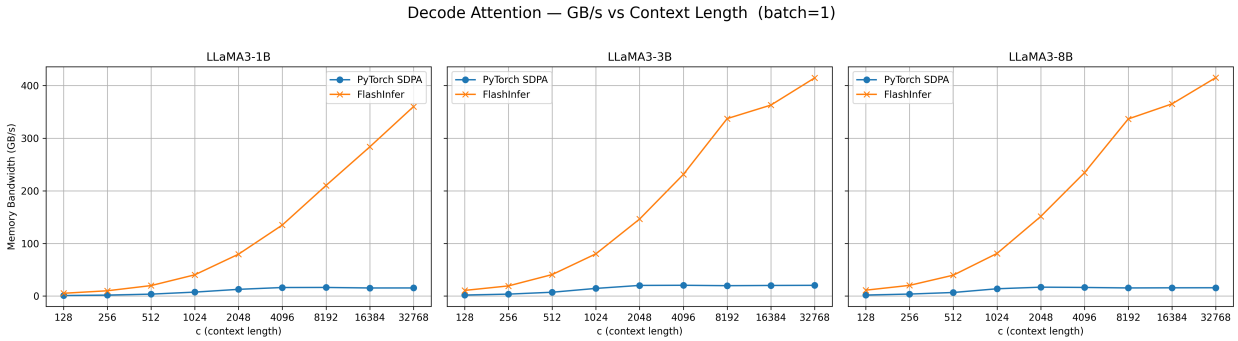

Figure 8: Memory bandwidth utilization (GB/s) for decode attention for LLaMA3-1B, LLaMA3-3B, LLaMA3-8B for a batch size of 1 and context lengths between 2^7 to 2^{15}.

ii. Batch size $= 2^0 - 2^6, c = 1024$ for all three models.

Our results are shown in Figure 9.

iii. Batch size = 128, c = 1024, page size = 1, 2, 4, 8, 16 for all three models using FlashInfer.

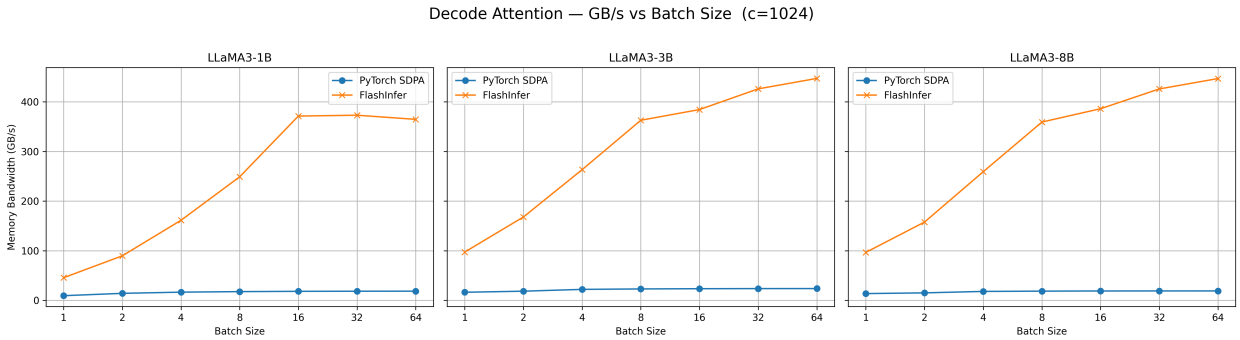

Figure 9: Memory bandwidth utilization (GB/s) for decode attention for LLaMA3-1B, LLaMA3-3B, LLaMA3-8B for a batch sizes from 1 to 32 and a context length of 1024.

Our results are shown in Figure 10.

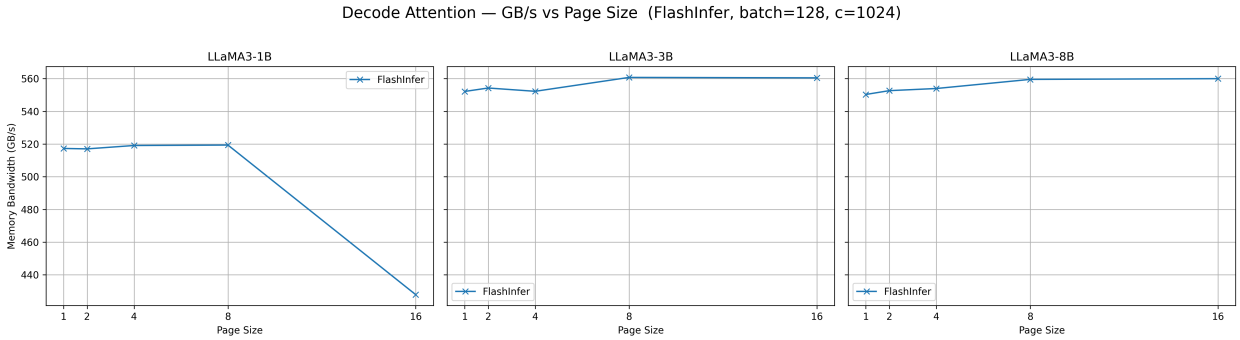

Figure 10: Memory bandwidth utilization (GB/s) for decode attention for LLaMA3-1B, LLaMA3-3B, LLaMA3-8B for a batch size of 128, a context length of 1024 and page sizes of 1, 2, 4, 8, and 16.

3 End-to-end Performance

Q1. Starting from the `Section3/flashinfer_pipeline.py`, construct a serving engine using FlashInfer attention and rope kernels with paged attention.

Q2. (a) With batch size 32, prefill length 256, decode length $2^5 - 2^{10}$, profile the prefill time and total decode time. Plot end-to-end time curve with the log(decode length) as the x-axis. Which phase is the key bottleneck? In the last decode cycle, which operation takes the longest time for different decode lengths?

Our results are shown in Figure 11. Decode takes significantly longer than prefill. The largest bottleneck is the prefill attention kernel provided by FlashInfer, which

used about 54% of compute time. With low amounts of decode lengths, GEMM and the attention kernel aare about tied for the most longest operation (both around 33% of time spent). However, as decode length increases, the attention kernel begins to take relatively much longer than GEMM.

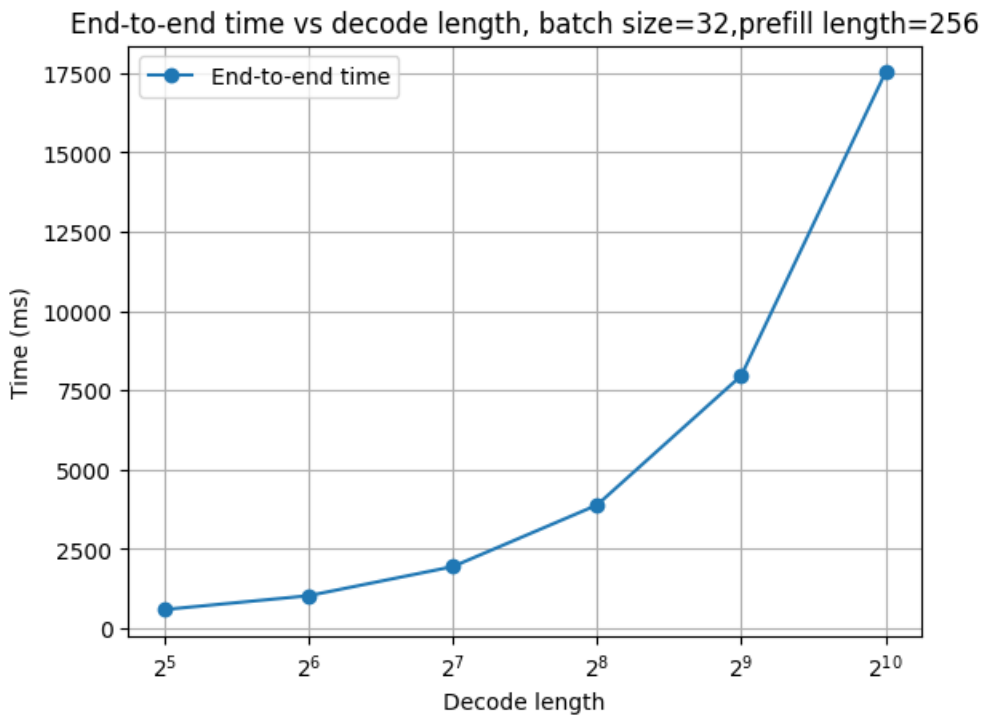

Figure 11: End-to-End time vs. Decode Length.

(b) With batch size 1, prefill length $2^8 - 2^{14}$, profile the prefill time. Plot end-to-end time curve with log(prefill length) as x-axis. Examine the prefill time breakdown. What operations are the dominating factor for various prefill lengths?

Our results are shown in Figure 12. At batch sizes $2^8 - 2^{10}$, GEMM operations are the dominating factor. However, at batch sizes $2^{11} - 2^{13}$, prefill attention kernels begin to take longer, and with batch size 2^{14}, the prefill attention kernel is dominating operation in terms of end-to-end time.

(c) With batch size $2^0 - 2^8$, prefill length 128, decode length 128, plot the end-to-end time curve with log(batch size) as x-axis. Plot the total throughput ((prefill + decode) / time) curve with log(batch size) as x-axis. When does the performance saturate?

Our results are shown in Figure 13. The model's performance begins to saturate at a batch size of 2^7, where the throughput starts to decrease, and the time noticeably begins to increase between 2^7 and 2^8.

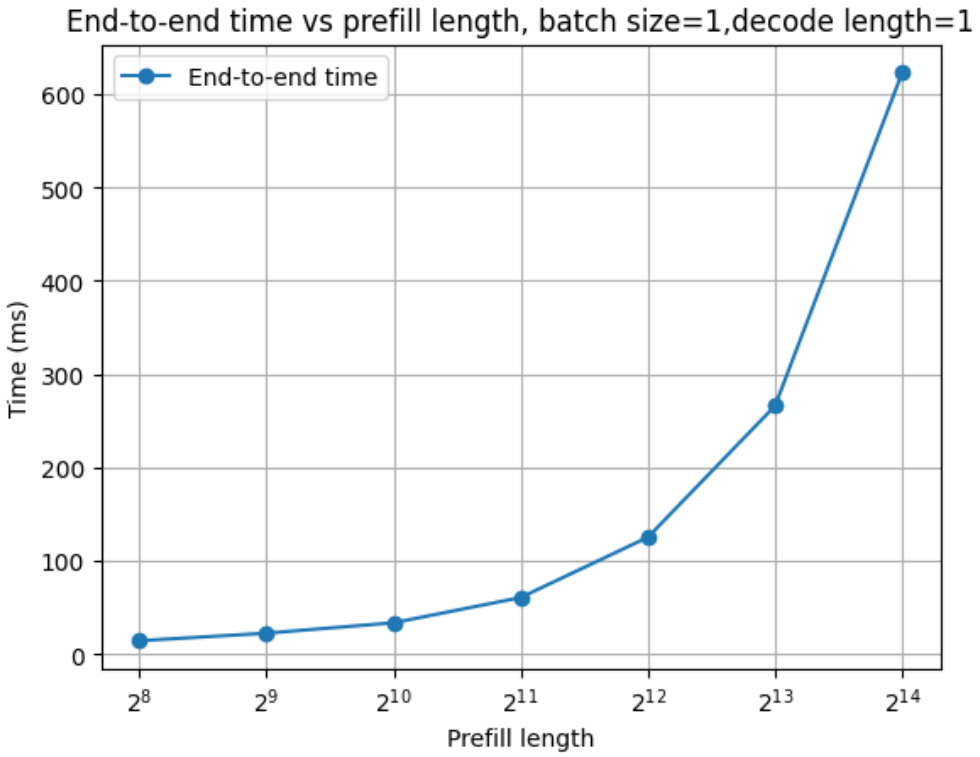

Figure 12: End-to-End time vs prefill length.

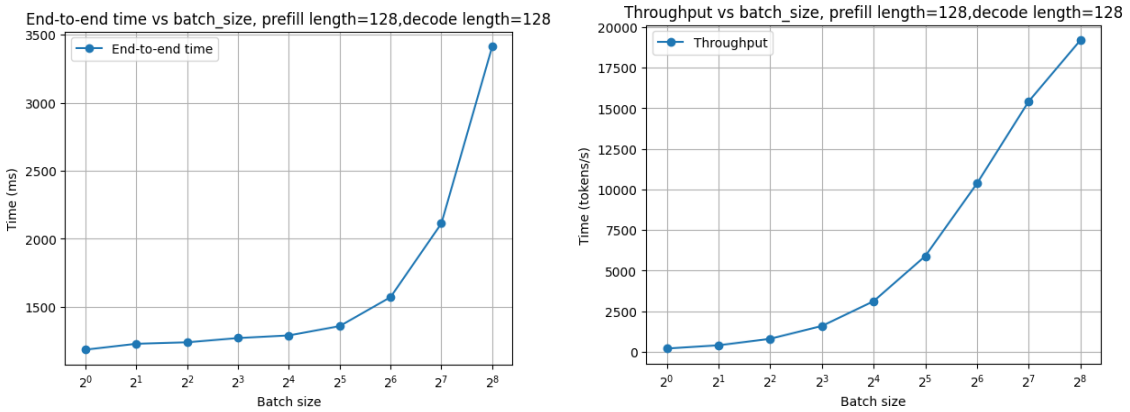

Figure 13: Left: End-to-End runtime vs batch size. Right: token throughput vs batch size.